

OASIS

OFFSHORE ADVANCED SIMULATION SOFTWARE

-

THEORY MANUAL

Sergio Fernández Ruano and Álvaro Rodríguez Luis

June 2019



Offshore Engineering and Ocean Energy Group
-
Environmental Hydraulics Institute of Cantabria

Contents

1	Introduction	3
2	Hydrostatics	4
2.1	Linear hydrostatics	4
2.2	Non-linear hydrostatics	4
3	Hydrodynamics	4
4	Mooring and towing lines	5
4.1	Equation	5
4.2	From PDE to ODE system	6
4.2.1	Weak formulation	7
4.2.2	Galerkin approximation	8
4.2.3	Spatial discretization and function base. Gauss-Lobatto-Lagrange polynomials	9
4.2.4	Derivatives and non-linear terms	12
4.2.5	Integrals and system matrices	14
4.3	Boundary conditions	16
4.3.1	Anchor, fairlead or actuator conditions	16
4.3.2	Multi-line joint conditions	17
4.3.3	Winch condition	18
4.4	Initial condition	20
5	Wind turbine	24
6	Oscillating water columns	24
7	Mechanical systems	24
8	Time integration	24
9	Coupling	24

1 Introduction

The purpose of this document is to describe in detail the theory behind the software developed by the Offshore Engineering and Ocean Energy Group of the Environmental Hydraulics Institute of Cantabria for offshore systems simulations and how to use it. The main goals that should be achieved developing this tool are:

- Develop a tool up to date with the literature, easy to use and with some additional capabilities that overcome problems that can not be solved with the commercial software available.
- Control a source code to be able to implement new features that may be necessary to simulate specific aspects of the devices studied in the group. In order to do this, the software must be modular, allowing to update it easily with new features constantly.
- Improve, put together and update all the codes developed by the group for the last years.

The main features of this software are:

- Floating body - sea interaction using linear potential theory, and CFD or experimental calibrations.
- Mooring and towing systems simulations.
- Multibody simulation, imposing the mechanical connections among the different bodies, allowing simulation of walls, pivots or joints.
- Wind turbines simulation.
- Oscillating water columns simulation.

The top improvements with respect to commercial software are:

- Instant position and second order for hydrodynamic forces on the bodies. Non-linear hydrostatics. Influence of bathymetry.
- High order finite elements method for mooring and towing lines, including bending and torsion effects, shock detection and diverse boundary conditions as winches (with dynamic positioning controllers) or complex bathymetry.
- Wind turbines integrated dynamically with the floating platform including aero-elasticity and rotor and pitch controllers.

The problem to be solved is a initial value problem with a set of subsystems interacting among them. Here a description of all the subsystems is presented together with the coupling techniques used and the time integration scheme.

2 Hydrostatics

2.1 Linear hydrostatics

(Sergio)

2.2 Non-linear hydrostatics

To be developed.

3 Hydrodynamics

(Sergio)

4 Mooring and towing lines

4.1 Equation

Considering that no external angular momentum is applied, the Equation 1 is used to model mooring and towing lines. This equation is the Newton's equation expressed per unit of line length.

$$\rho_0 \frac{\partial^2 \mathbf{r}(t, s)}{\partial t^2} = \frac{\partial \mathbf{F}(t, s)}{\partial s} + \mathbf{f}(t, s) \left| \frac{\partial \mathbf{r}}{\partial s} \right| \quad (1)$$

With L being cable length, ρ_0 cable mass per unit of length, $s \in [0, L]$ arc-length parameter, $\mathbf{r} : [t_0, \infty) \times [0, L] \rightarrow \mathbb{R}^3$ position and $\mathbf{t} : [t_0, \infty) \times [0, L] \rightarrow \mathbb{R}^3$ tangent vector. $\mathbf{F} : [t_0, \infty) \times [0, L] \rightarrow \mathbb{R}$ is the internal forces vector and $\mathbf{f} : [t_0, \infty) \times [0, L] \rightarrow \mathbb{R}^3$ the external forces per unit of length vector.

For the external forces term, the hydrodynamic forces and the hydrostatic forces (first four terms on Equation 2) are based on (6) and floor interaction forces (last two terms on Equation 2) where developed for this project, **considering a flat ocean floor with no horizontal friction**. The considered external forces are shown in Equation 2.

$$\mathbf{f} = \mathbf{f}_{hg} + \mathbf{f}_{dt} + \mathbf{f}_{dn} + \mathbf{f}_{mt} + \mathbf{f}_{mn} + \mathbf{f}_{gz} + \mathbf{f}_{gx} + \mathbf{f}_{gy} \quad (2)$$

Where:

$\mathbf{f}_{hg} = \frac{\rho_0 - \rho_w \cdot A}{e + 1} \mathbf{g}$	Gravity and buoyancy forces.
$\mathbf{f}_{dt} = -\frac{1}{2} \cdot C_{DT} \cdot d \cdot \rho_w \cdot \mathbf{v}_t \cdot \mathbf{v}_t$	Tangential drag force.
$\mathbf{f}_{dn} = -\frac{1}{2} \cdot C_{DN} \cdot d \cdot \rho_w \cdot \mathbf{v}_n \cdot \mathbf{v}_n$	Normal drag force.
$\mathbf{f}_{mt} = -C_{MT} \cdot \frac{\pi d^2}{4} \rho_w \cdot \mathbf{a}_t$	Tangential added mass force.
$\mathbf{f}_{mn} = -C_{MN} \cdot \frac{\pi d^2}{4} \rho_w \cdot \mathbf{a}_n$	Normal added mass force.
$\mathbf{f}_{gz} = \mathbf{z} \cdot \mathbf{f}_{hg} \cdot e^{-G_K \cdot d \cdot (r_z - z_f) / \mathbf{f}_{hg} }$	Normal ground reaction force.
$\mathbf{f}_{gvz} = \mathbf{f}_{hg} \cdot G_d \cdot d \cdot \min(v_z, 0)^2$	Normal ground damping force.

The parameters in the previous expressions are defined as:

ρ_w	water density,	$\mathbf{v}_t$	line tangential velocity,
$\mathbf{g}$	gravity acceleration vector	$\mathbf{v}_n$	line normal velocity,
A	line section,	$\mathbf{a}_t$	line tangential acceleration,
d	line diameter,	$\mathbf{a}_n$	line normal acceleration,
e	axial strain,	G_K	ground normal stiffness
C_{DT}	line tangential drag coefficient,	G_c	ground's critical damping,
C_{DN}	line normal drag coefficient,	$\mathbf{z}$	vertical unitary vector,
C_{MT}	line tangential added mass coefficient,	r_z	vertical coordinate of a line node,
C_{MN}	line normal added mass coefficient,	z_f	vertical coordinate of ocean floor and
		v_z	vertical velocity of a line node.

Tangent velocity is computed as $\mathbf{v}_t = (\mathbf{v} \cdot \mathbf{t}) \cdot \mathbf{t}$, where $\mathbf{t}$ is the unitary tangent vector. Normal velocity is $\mathbf{v}_n = \mathbf{v} - \mathbf{v}_t$. The same applies for acceleration.

For the internal forces term, **considering negligible the effects of torsion and bending**, the expression is shown in Equation 3.

$$\mathbf{F} = \mathbf{t} \cdot T \quad (3)$$

Where, using the notation $f' = \frac{\partial f}{\partial s}$ and $\dot{f} = \frac{\partial f}{\partial t}$

$\mathbf{t} = \frac{\mathbf{r}'}{ \mathbf{r}' }$	unitary tangent vector,
$T = E \cdot A \cdot (e + \beta \cdot \dot{e})$	tension term according to Rayleigh tension model (3).
E	line Young modulus,
β	line internal damping coefficient,
$e = \mathbf{r}' - 1$	axial strain and
$\dot{e} = \frac{\mathbf{r}' \cdot \dot{\mathbf{r}}'}{ \mathbf{r}' } = \mathbf{t} \cdot \dot{\mathbf{r}}'$	temporal derivative of line axial strain.

The presented tension model is a linear symmetric model, but other models can be used such as linear tension only model ($T = \max\{0, T\}$, already implemented) **or non-linear tension models**.

4.2 From PDE to ODE system

The equation in the previous section is a highly non-linear wave equation, a partial differential equation that can be expressed in the form:

$$\ddot{\mathbf{r}} = \Phi(\dot{\mathbf{r}}, \mathbf{r}, t)$$

where Φ is a non-linear function where the term $\mathbf{r}''$ appears. In order to be able to obtain the time evolution of a line, it would be convenient to transform this equation in a initial value problem of the form:

$$\begin{cases} \frac{\partial \mathbf{u}}{\partial t} = f(\mathbf{u}, t) \\ \mathbf{u}(0) = \mathbf{u}_0 \end{cases}$$

With f being the ODE function, $\mathbf{u}_0$ known initial condition, and $\mathbf{u}$ the unknown. In this case the unknown will be:

$$\mathbf{u} = \begin{pmatrix} \mathbf{r} \\ \mathbf{v} \end{pmatrix}$$

where $\mathbf{r}$ and $\mathbf{v}$ are vectors that contain, respectively, the positions and velocities of different points in the line. This way:

$$\frac{\partial \mathbf{u}}{\partial t} = \begin{pmatrix} \mathbf{v} \\ \mathbf{a} \end{pmatrix}$$

where $\mathbf{a}$ is the vector containing the accelerations of the points. The idea is to use high order finite elements method to compute a matrix $\mathbf{A}$ and a vector $\mathbf{B}$ such that:

$$\begin{pmatrix} \mathbf{v} \\ \mathbf{a} \end{pmatrix} = \frac{\partial \mathbf{u}}{\partial t} = f(\mathbf{u}, t) = f\left(\begin{pmatrix} \mathbf{r} \\ \mathbf{v} \end{pmatrix}, t\right) = \begin{pmatrix} \mathbf{v} \\ \mathbf{A}^{-1} \cdot \mathbf{B} \end{pmatrix} \quad (4)$$

The vector $\mathbf{B}$, as well as other vectors that will appear in this section, such as the position, velocity and acceleration vectors ($\mathbf{r}$, $\dot{\mathbf{r}}$ and $\ddot{\mathbf{r}}$) or the internal and external forces vectors ($\mathbf{F}$ and $\hat{\mathbf{f}}$), may not be a single column vector, but a matrix of size $(N \times 3)$, in which each column represents one of the Cartesian coordinates (x , y and z), since the positions and forces belong to $\mathbb{R}^3$. Later, to couple the lines with the rest of degrees of freedom of the problem (bodies, wind turbine...) it will be necessary to reshape these matrices into columns vectors. From now on, in this document, the $(N \times 3)$ matrices will be called vectors. The first step to find $\mathbf{A}$ and $\mathbf{B}$ is to find the weak formulation of the problem.

4.2.1 Weak formulation

The boundary conditions on the lines will be the positions of both ends. This is a Dirichlet boundary condition and consequently, the test function $\mathbf{w}$ used for the weak formulation belongs to the space of functions:

$$V = \{f \in C_1 \mid f(0) = f(L) = 0\}$$

The line equation is multiplied with the test function and taking the integral with s along the line length:

$$\begin{aligned} \rho_0 \ddot{\mathbf{r}} &= \mathbf{F}' + \mathbf{f} |\mathbf{r}'| \Rightarrow \\ \int_0^L \rho_0 \ddot{\mathbf{r}} \mathbf{w} ds &= \int_0^L \mathbf{F}' \mathbf{w} ds + \int_0^L \mathbf{f} \cdot \left| \frac{\partial \mathbf{r}}{\partial s} \right| \mathbf{w} ds \end{aligned}$$

Finally, using integration by parts and the properties of $\mathbf{w}$ we obtain the weak formulation of the problem:

$$\begin{aligned} \int_0^L \rho_0 \ddot{\mathbf{r}} \mathbf{w} ds &= [\mathbf{F} \mathbf{w}]_0^L - \int_0^L \mathbf{F} \mathbf{w}' ds + \int_0^L \mathbf{f} \cdot \left| \frac{\partial \mathbf{r}}{\partial s} \right| \mathbf{w} ds = \\ &= - \int_0^L \mathbf{F} \mathbf{w}' ds + \int_0^L \mathbf{f} \cdot \left| \frac{\partial \mathbf{r}}{\partial s} \right| \mathbf{w} ds \end{aligned} \quad (5)$$

Equation 5 allows using Galerkin approximation to transform the PDE in a ODE system.

4.2.2 Galerkin approximation

Considering a finite sub-base of V , $\{\phi_i\}_{i=0}^{N-1}$, the solution can be written as a combination of the base functions:

$$\mathbf{r} = \sum_{i=0}^{N-1} \mathbf{r}_i \phi_i$$

where it is important to notice that the coefficients $\{\mathbf{r}_i\}_{i=0}^{N-1}$ are vectors in $\mathbb{R}^3$. Also, for the derivatives:

$$\frac{\partial^{m+n} \mathbf{r}}{\partial t^m \partial s^n} = \sum_{i=0}^N \frac{\partial^m \mathbf{r}_i}{\partial t^m} \frac{\partial^n \phi_i}{\partial s^n}$$

With a good choice of the base and knowing the positions and velocities of certain points in the line, the derivatives needed compute internal forces $\mathbf{F}$ and external forces $\mathbf{f}$ can be easily obtained, as it is shown later. Considering this, and taking $\hat{\mathbf{f}} = \mathbf{f} \cdot \left| \frac{\partial \mathbf{r}}{\partial s} \right|$, internal and external forces can be expressed in terms of the base, as it was done for $\mathbf{r}$:

$$\mathbf{F} = \sum_{i=0}^{N-1} \mathbf{F}_i \phi_i \quad \hat{\mathbf{f}} = \sum_{i=0}^{N-1} \hat{\mathbf{f}}_i \phi_i$$

Now, using the functions in the base as test functions, the following equations are obtained.

$$\rho_0 \int_0^L \left(\sum_{i=0}^{N-1} \ddot{\mathbf{r}}_i \phi_i \right) \phi_j ds = - \int_0^L \left(\sum_{i=0}^{N-1} \mathbf{F}_i \phi_i \right) \phi'_j ds + \int_0^L \left(\sum_{i=0}^{N-1} \hat{\mathbf{f}}_i \phi_i \right) \phi_j ds \quad j = 0, 1, \dots, N-1 \quad (6)$$

Applying the integrals linearity:

$$\rho_0 \sum_{i=0}^{N-1} \ddot{\mathbf{r}}_i \int_0^L \phi_i \phi_j ds = - \sum_{i=0}^{N-1} \mathbf{F}_i \int_0^L \phi_i \phi'_j ds + \sum_{i=0}^{N-1} \hat{\mathbf{f}}_i \int_0^L \phi_i \phi_j ds \quad j = 0, 1, \dots, N-1 \quad (7)$$

Finally, using matrix notation:

$$\rho_0 \mathbf{M} \begin{pmatrix} \ddot{\mathbf{r}}_0 \\ \ddot{\mathbf{r}}_1 \\ \vdots \\ \ddot{\mathbf{r}}_{N-1} \end{pmatrix} = -\mathbf{K}^* \begin{pmatrix} \mathbf{F}_0 \\ \mathbf{F}_1 \\ \vdots \\ \mathbf{F}_{N-1} \end{pmatrix} + \mathbf{M} \begin{pmatrix} \hat{\mathbf{f}}_0 \\ \hat{\mathbf{f}}_1 \\ \vdots \\ \hat{\mathbf{f}}_{N-1} \end{pmatrix} \quad (8)$$

$$\text{where:} \quad \mathbf{M}_{ij} = \int_0^L \phi_i \phi_j ds \quad \mathbf{K}_{ij}^* = \int_0^L \phi'_i \phi_j ds \quad (9)$$

In order to compute $\mathbf{M}_{ij}$, $\mathbf{K}_{ij}^*$, $\mathbf{F}_i$ and $\hat{\mathbf{f}}_i$, first it is necessary to describe the function base.

4.2.3 Spatial discretization and function base. Gauss-Lobatto-Lagrange polynomials

The spatial domain is the interval $[0, L] \in \mathbb{R}$ and it is discretized with N points $\{s_i\}_{i=0}^{N-1}$. There are several characteristics desirable in the discretization and the function base:

- The discretization should be as regular as possible and it must meet $s_0 = 0$ and $s_{N-1} = L$, so it is easy to impose boundary conditions.
- The base functions should meet that $\phi_i(s_j) = \delta_{ij}$. This way, the expansion vector coefficients r_i have a physical meaning: the position of node i .
- The base functions must be good interpolation functions, with good accuracy and avoiding Runge effect (to avoid Runge effect with a polynomial basis, choosing polynomials of order higher than 5 should be avoided). Also, they should yield good conditioned matrices $\mathbf{M}$ and $\mathbf{K}^*$.

In this case, the domain is divided in $n - 1$ elements of equal length, and at each element the Gauss-Lobatto quadrature nodes of order p ($\{\xi_i\}_{i=0}^p$), are considered by mapping the nodes from the interval $[-1, 1]$ to the corresponding element. Among all the quadrature rules available, the Gauss-Lobatto quadrature presents the advantage that -1 and 1 belong to the quadrature nodes for any order, and this implies that, in the discretization proposed, two contiguous elements will always share a node, simplifying the continuity conditions. Figure 1 shows the Gauss-Lobatto nodes. Finally, the base functions are locally described as the Lagrange interpolation polynomials over each Gauss-Lobatto node, these is the Gauss-Lobatto-Lagrange polynomials, which are widely accepted in the literature (2). These polynomials can be classified in two sets, as shown in Figure 2 for the fifth order case. For generic order p , the first set are the two polynomials, θ_0 and θ_p , that are non-zero in either -1 or 1 , and the second set are the $p - 1$ polynomials, θ_1 to θ_{p-1} , that are zero in both -1 and 1 .

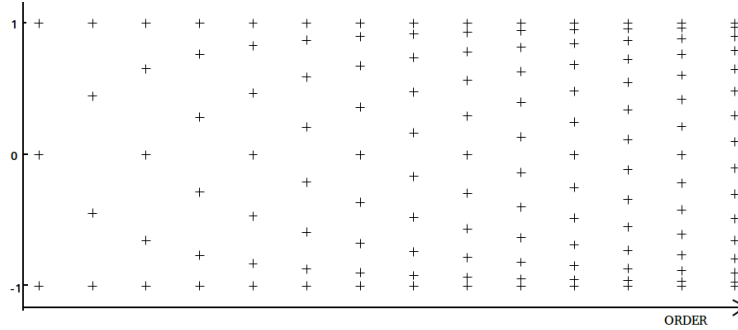


Figure 1: Gauss-Lobatto points for order $p = 2, \dots, 15$.

The continuous Galerkin scheme is applied, implying that, globally, for two contiguous elements k and $k + 1$, the polynomial θ_p in the element k and the polynomial θ_0 in element $k + 1$, are considered as one function. This function, known as vertex function and defined in $[0, L]$, is zero in all the domain except in the element k where it takes the values of θ_p and in element $k + 1$ where it takes the values of θ_0 . On the other hand we have the bubble functions, that are defined as zero in all the domain except for on element, where they take the value of one of the polynomials θ_1 to θ_{p-1} . Both vertex and bubble functions are shown in Figure 3.

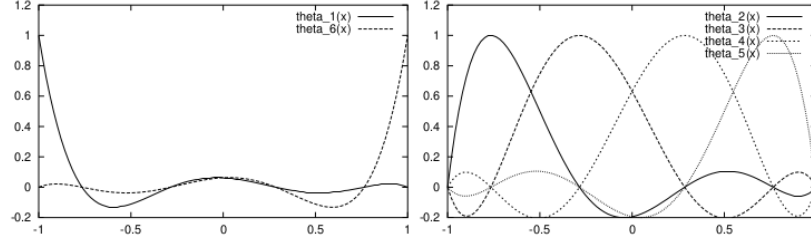


Figure 2: Fifth order Gauss-Lobatto-Lagrange polynomials.

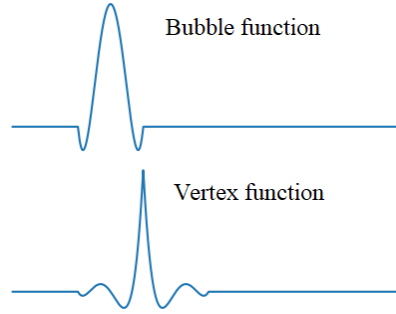


Figure 3: Descriptive scheme of vertex and bubble functions.

The set of all vertex and bubble functions compose the chosen base. For example for the case of three elements of order three, in Figure 4 all the base functions are shown. There are two special vertex functions, ϕ_0 and ϕ_{N-1} , they are only half of the usual vertex functions. In fact, they do not belong to the space of functions V , as they are not zero at $s = 0$ and $s = L$. This only affects the first and last equations of the system, where the term $\mathbf{\Gamma}$ that does not vanish in the integration by parts must be added, this term is:

$$\mathbf{\Gamma} = \begin{pmatrix} -\mathbf{F}_0 \\ 0 \\ \vdots \\ 0 \\ \mathbf{F}_{N-1} \end{pmatrix} \quad (10)$$

In summary and adding some notation, this are the important aspects of the spatial discretization:

- p is the order of the polynomials.
- $n - 1$ is the number of elements and $h = \frac{L}{n-1}$ is the length of each element.
- $N = n + (n - 1)(p - 1) = p(n - 1) + 1$ is the number of base functions¹.
- $s_i = h \left(\frac{i - (i \% p)}{p} + \frac{1 + \xi_{(i \% p)}}{2} \right)$ are the discretization nodes².

¹Note that n is the number of vertex functions and $(n - 1)(p - 1)$ is the number of bubble functions.

²Here $(a \% b)$ denotes the $(a \bmod b)$, and the reader should note that $\frac{i - (i \% p)}{p}$ is the element index and $\frac{1 + \xi_{(i \% p)}}{2}$ maps the quadrature nodes from $[-1, 1]$ to $[0, 1]$.

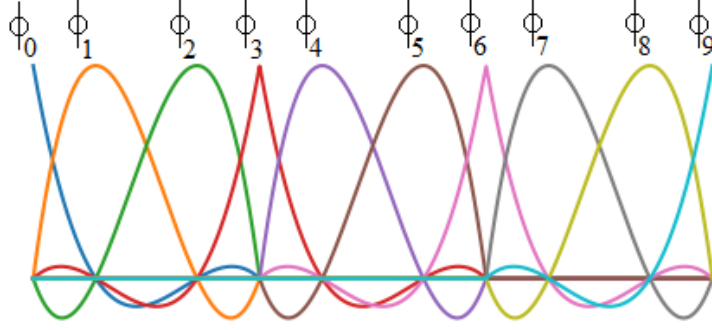


Figure 4: Example of base functions for three third order elements.

For the base choice, the formal formulation would be as follows.

$$\phi_i(s) = \begin{cases} \alpha_i(s) & \text{if } (i \% p) = 0 \\ \beta_i(s) & \text{else} \end{cases}$$

where $\alpha_i(s)$ is the vertex function:

$$\alpha_i(s) = \begin{cases} \begin{cases} \theta_0(2 \cdot \frac{s-s_i}{h} - 1) & \text{if } s_i \leq s \leq s_{i+p} \\ 0 & \text{else} \end{cases} & \text{if } i = 0 \\ \begin{cases} \theta_p(2 \cdot \frac{s-s_{i-p}}{h} - 1) & \text{if } s_{i-p} \leq s < s_i \\ 0 & \text{else} \end{cases} & \text{if } i = N - 1 \\ \begin{cases} \theta_0(2 \cdot \frac{s-s_i}{h} - 1) & \text{if } s_i \leq s \leq s_{i+p} \\ \theta_p(2 \cdot \frac{s-s_{i-p}}{h} - 1) & \text{if } s_{i-p} \leq s < s_i \\ 0 & \text{else} \end{cases} & \text{else} \end{cases}$$

and $\beta_i(s)$ is the bubble function:

$$\beta_i(s) = \begin{cases} \theta_{(i \% p)}(2 \cdot \frac{s-s_{p \cdot k}}{h} - 1) & \text{if } s_{p \cdot k} \leq s \leq s_{p \cdot (k+1)} \\ 0 & \text{else} \end{cases}$$

where $k = \frac{i-(i \% p)}{p}$ is the element index. The base functions are continuous, but their derivatives are not, because of this it is necessary to take a special care when computing derivatives using these functions, and that is done in the next section.

4.2.4 Derivatives and non-linear terms

With the current basis, if the positions of the discretization nodes, $(r_0, r_1, \dots, r_{N-1})^T$, are known, the derivatives at any point of the chain can be computed with:

$$\frac{\partial^n \mathbf{r}}{\partial s^n}(s) = \sum_{i=0}^{N-1} r_i \frac{\partial^n \phi_i}{\partial s^n}(s) = \left(\frac{\partial^n \phi_0}{\partial s^n}(s), \frac{\partial^n \phi_1}{\partial s^n}(s), \dots, \frac{\partial^n \phi_{N-1}}{\partial s^n}(s) \right) \cdot \begin{pmatrix} r_0 \\ r_1 \\ \vdots \\ r_{N-1} \end{pmatrix}$$

In general, it is interesting to compute the derivatives at the discretization nodes, in matrix notation, this can be done with the following expression:

$$\begin{pmatrix} \frac{\partial^n \mathbf{r}}{\partial s^n}(s_0) \\ \frac{\partial^n \mathbf{r}}{\partial s^n}(s_1) \\ \vdots \\ \frac{\partial^n \mathbf{r}}{\partial s^n}(s_{N-1}) \end{pmatrix} = \mathbf{D}_n \begin{pmatrix} r_0 \\ r_1 \\ \vdots \\ r_{N-1} \end{pmatrix}$$

where:

$$\mathbf{D}_n = \begin{pmatrix} \frac{\partial^n \phi_0}{\partial s^n}(s_0) & \frac{\partial^n \phi_1}{\partial s^n}(s_0) & \dots & \frac{\partial^n \phi_{N-1}}{\partial s^n}(s_0) \\ \frac{\partial^n \phi_0}{\partial s^n}(s_1) & \frac{\partial^n \phi_1}{\partial s^n}(s_1) & \dots & \frac{\partial^n \phi_{N-1}}{\partial s^n}(s_1) \\ \vdots & \vdots & & \vdots \\ \frac{\partial^n \phi_0}{\partial s^n}(s_{N-1}) & \frac{\partial^n \phi_1}{\partial s^n}(s_{N-1}) & \dots & \frac{\partial^n \phi_{N-1}}{\partial s^n}(s_{N-1}) \end{pmatrix}$$

and for the points where there is a discontinuity in the derivative, the following approximation is used:

$$\frac{\partial^n \phi_i}{\partial s^n}(s_j) = \frac{1}{2} \left(\frac{\partial^n \phi_i}{\partial s^n}(s_j^-) + \frac{\partial^n \phi_i}{\partial s^n}(s_j^+) \right)$$

The global derivation matrix of order n , $\mathbf{D}_n$ is constant in time and it is computed only once, unless the line length changes, in which case the matrix can be updated with a scalar multiplication. It can be assembled using the local derivation matrix, $\mathbf{D}_n^L$, defined as $(\mathbf{D}_n^L)_{ij} = \frac{\partial^n \theta_j}{\partial x^n}(\xi_i)$, using the following algorithm.

$$\mathbf{D}_n = \mathbf{0}_{N \times N}$$

for $(k = 0 : n - 1)\{$
 $\quad \mathbf{D}_n[k \cdot p : (k + 1) \cdot p, \quad k \cdot p : (k + 1) \cdot p] = \mathbf{D}_n[k \cdot p : (k + 1) \cdot p, \quad k \cdot p : (k + 1) \cdot p] + \mathbf{D}_n^L$
 $\}$

for $(k = 1 : n - 1)\{$
 $\quad \mathbf{D}_n[k \cdot p : (k + 1) \cdot p, \quad :] = \mathbf{D}_n[k \cdot p : (k + 1) \cdot p, \quad :] \cdot \frac{1}{2}$
 $\}$

$$\mathbf{D}_n = \left(\frac{2}{l}\right)^n \cdot \mathbf{D}_n$$

In this algorithm, first the global derivation matrix is initialized and then the values of the local derivation matrix are inserted in the global matrix with a first loop over all elements, as shown in Figure 5 for a case with four elements of order three. In a second loop, the rows of the matrix that relate to nodes where the base functions derivatives present discontinuities are divided by two, as shown in Figure 6 (the values of the derivatives at both sides of the node were already summed in the previous loop). Finally, the full matrix is multiplied by the factor $\left(\frac{2}{l}\right)^n$, that is necessary because the chain rule of derivation is being applied with the mapping from $[-1, 1]$ to $[s_k, s_{k+1}]$ at every element k .

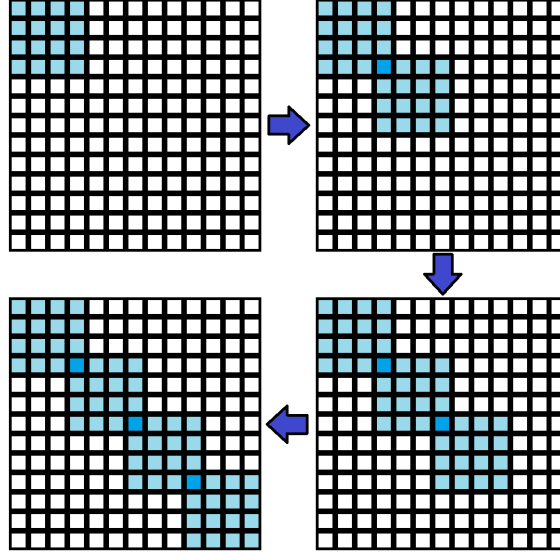


Figure 5: Assembling step. In the top left corner of the figure, only the first local matrix $\mathbf{D}_n^L$ has been added to the full matrix. In the dark blue elements, the values of $\mathbf{D}_n^L[0, 0]$ and $\mathbf{D}_n^L[p, p]$ are being summed.

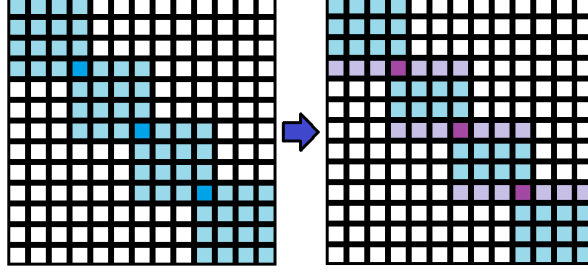


Figure 6: Discontinuity of derivatives treatment. The purple rows are divided by two to average the value of the derivative at both sides of de node.

If the line nodes velocities are known, the crossed derivatives at the nodes can also be computed easily using the global derivation matrix:

$$\begin{pmatrix} \frac{\partial^n \mathbf{r}}{\partial s^n \partial t}(s_0) \\ \frac{\partial^n \mathbf{r}}{\partial s^n \partial t}(s_1) \\ \vdots \\ \frac{\partial^n \mathbf{r}}{\partial s^n \partial t}(s_{N-1}) \end{pmatrix} = \mathbf{D}_n \begin{pmatrix} \frac{\partial r_0}{\partial t} \\ \frac{\partial r_1}{\partial t} \\ \vdots \\ \frac{\partial r_{N-1}}{\partial t} \end{pmatrix}$$

Once all the derivatives are known, it is easy to compute all the non-linear terms that appear in the expressions of $\mathbf{F}$ and $\hat{\mathbf{f}}$:

- $\mathbf{t}(s_i) = \frac{\mathbf{r}'(s_i)}{|\mathbf{r}'(s_i)|}$
- $e(s_i) = |\mathbf{r}'(s_i)| - 1$
- $\dot{e}(s_i) = \mathbf{t}(s_i) \cdot \dot{\mathbf{r}}'(s_i)$

Although directly computing second or higher order spatial derivatives is possible with this method, the error increases drastically when this is done, and it should always be avoided. For example, in this case, using the weak formulation and integration by parts, it was possible to change a product of a second order derivative with a function for a product of two first order derivatives.

Introducing first order derivatives, which also have a small error, in non-linear expressions where parameters of order up to 10^8 appear, may produce an error that affects significantly the solution, specially when the interpolated functions are not smooth. **For this reason, it would be interesting to perform an analysis of the error, in order to know which is the roughest discretization required to obtain a reasonable solution.**

4.2.5 Integrals and system matrices

In general, for a continuous function $f(x)$ defined in $[-1, 1]$, Gauss-Lobatto quadrature of order p can be applied to compute the integral:

$$\int_{-1}^1 f(x) dx = \sum_{i=0}^p \omega_i \cdot f(\xi_{i,p-1}) + E$$

where E is the error of the quadrature rule and the Gauss-Lobatto weights $\{\omega_i\}$ and nodes $\{\xi_{i,p-1}\}$ can be computed with the method detailed in Karniadakis (2) book, page 61. If $f(x)$ is a polynomial of order less or equal to $2 \cdot p - 1$, then the quadrature is exact ($E = 0$).

If a discretization of order p is chosen for the studied line, a quadrature rule of order at least $p + 1$ is necessary to evaluate exactly³ the integrals that appear in Equation 9. In each element of the discretization, the same local function base is used, and then, the elementary matrices can be used to assemble the full matrices. For element k :

$$\begin{aligned}\mathbf{M}_{ij}^{(k)} &= \int_{s_{p \cdot k}}^{s_{p \cdot (k+1)}} \theta_i\left(\frac{2}{h}(s - s_{p \cdot k}) - 1\right) \cdot \theta_j\left(\frac{2}{h}(s - s_{p \cdot k}) - 1\right) ds = \left(\frac{h}{2}\right) \cdot \int_{-1}^1 \theta_i(x) \cdot \theta_j(x) dx \\ \mathbf{K}_{ij}^{*(k)} &= \int_{s_{p \cdot k}}^{s_{p \cdot (k+1)}} \left(\theta_i\left(\frac{2}{h}(s - s_{p \cdot k}) - 1\right)\right)' \cdot \theta_j\left(\frac{2}{h}(s - s_{p \cdot k}) - 1\right) ds = \\ &= \int_{s_{p \cdot k}}^{s_{p \cdot (k+1)}} \left(\frac{h}{2}\right) \cdot \theta_i'\left(\frac{2}{h}(s - s_{p \cdot k}) - 1\right) \cdot \theta_j\left(\frac{2}{h}(s - s_{p \cdot k}) - 1\right) ds = \\ &= \left(\frac{h}{2}\right) \cdot \int_{-1}^1 \left(\frac{2}{h}\right) \cdot \theta_i'(x) \cdot \theta_j(x) dx = \int_{-1}^1 \theta_i'(x) \cdot \theta_j(x) dx\end{aligned}$$

These elementary matrices are the same for every element, we define constant elementary matrices $\mathbf{M}^{(e)}$ and $\mathbf{K}^{*(e)}$ as $\mathbf{M}_{ij}^{(e)} = \left(\frac{h}{2}\right) \cdot \int_{-1}^1 \theta_i(x) \cdot \theta_j(x) dx$ and $\mathbf{K}_{ij}^{*(e)} = \int_{-1}^1 \theta_i'(x) \cdot \theta_j(x) dx$. The following algorithm is used for assembling of the full matrices:

```

M = 0N×N
K* = 0N×N

for (k = 0 : n - 2){
  M[k · p : (k + 1) · p + 1, k · p : (k + 1) · p + 1] =
    = M[k · p : (k + 1) · p + 1, k · p : (k + 1) · p + 1] + M(e)
  K*[k · p : (k + 1) · p + 1, k · p : (k + 1) · p + 1] =
    = K*[k · p : (k + 1) · p + 1, k · p : (k + 1) · p + 1] + K*(e)
}

```

This algorithm is the same as the one used for the derivation matrix in Figure 5, but without dividing by two, since there are no discontinuities in this case. Also, the elementary matrices are computed using Gauss-Lobatto Gaussian quadrature with the following algorithm:

```

M(e) = 0N×N
K*(e) = 0N×N
for (i = 0 : p){
  for (j = 0 : p){
    for (k = 0 : p + 1){
      M(e)[i, j] = M(e)[i, j] +  $\left(\frac{h}{2}\right) \cdot \omega_k \cdot \theta_i(\xi_{k,p}) \cdot \theta_j(\xi_{k,p})$ 
      K*(e)[i, j] = K*(e)[i, j] +  $\omega_k \cdot \theta_i'(\xi_{k,p}) \cdot \theta_j(\xi_{k,p})$ 
    }
  }
}

```

³If the order of the discretization is p , the order of the product $\phi_i \phi_j$ is $2 \cdot p$, and a quadrature of order p would be insufficient to compute the integral of this product exactly.

In this algorithm there are three nested loops, the first one over the matrix rows, the second one over the matrix columns and the last one is over the quadrature nodes. The system matrices are constant in time and they are computed only once, unless for the matrix $\mathbf{M}$, and only if the line length changes, in which case the matrix can be updated with a scalar multiplication.

Recall that the purpose of this whole section was to compute a matrix $\mathbf{A}$ and a vector $\mathbf{B}$ such that:

$$\begin{pmatrix} \mathbf{v} \\ \mathbf{a} \end{pmatrix} = \frac{\partial \mathbf{u}}{\partial t} = f(\mathbf{u}, t) = f\left(\begin{pmatrix} \mathbf{r} \\ \mathbf{v} \end{pmatrix}, t\right) = \begin{pmatrix} \mathbf{v} \\ \mathbf{A}^{-1} \cdot \mathbf{B} \end{pmatrix}$$

So far, the result is

$$\mathbf{A} = \mathbf{M}$$

and

$$\mathbf{B} = -\mathbf{K}^* \cdot \mathbf{F} + \mathbf{M} \cdot \hat{\mathbf{f}} + \mathbf{\Gamma}$$

where the terms $\mathbf{M}$, $\mathbf{K}^*$, $\mathbf{F}$ and $\hat{\mathbf{f}}$ have been described in detail.

Note that the final expression for computing the acceleration at the nodes of the chain is the system $\mathbf{M} \cdot \mathbf{a} = -\mathbf{K}^* \cdot \mathbf{F} + \mathbf{M} \cdot \hat{\mathbf{f}}(\mathbf{a}) + \mathbf{\Gamma}$, where $\hat{\mathbf{f}}$ depends on $\mathbf{a}$ because the added mass forces are part of the external force. This problem is solved using an iterative method where the following algorithm is followed, where "tol" is the tolerance of the iterative method and "nIterMax" is the maximum number of iterations, introduced to limit the computational cost.

```

set  $\mathbf{a}^0$  as the acceleration at previous time-step
solve  $\mathbf{M} \cdot \mathbf{a}^1 = -\mathbf{K}^* \cdot \mathbf{F} + \mathbf{M} \cdot \hat{\mathbf{f}}(\mathbf{a}^0) + \mathbf{\Gamma}$ 
 $k = 1$ 

while (( $\|\mathbf{a}^k - \mathbf{a}^{k-1}\| > \text{tol}$ ) & ( $k < \text{nIterMax}$ )) {
    solve  $\mathbf{M} \cdot \mathbf{a}^{k+1} = -\mathbf{K}^* \cdot \mathbf{F} + \mathbf{M} \cdot \hat{\mathbf{f}}(\mathbf{a}^k) + \mathbf{\Gamma}$ 
     $k = k + 1$ 
}
 $\mathbf{a} = \mathbf{a}^k$ 

```

The problem is not completely solved yet. First, it is necessary to describe how to impose the boundary conditions and the initial condition. This is done in the following section.

4.3 Boundary conditions

Once the matrix $\mathbf{A}$ and the vector $\mathbf{B}$ in Equation 4 are computed, it is necessary to modify them before solving the linear system in order to impose the boundary conditions. In this section, it is shown how to do this for the different options considered.

4.3.1 Anchor, fairlead or actuator conditions

When the line is connected to an anchor, a floating body fairlead or a lab actuator, the position, velocity and acceleration of that point of the line are known, and must be imposed. To do so, for

computing the internal and external forces vectors, which depend on the position and the velocity of the line, the boundary data is considered in the calculations. Then, before solving the system, first row, last row or both rows of the system matrix are set to zero, except for the element in the diagonal, and in the forces vector the first row, the last row or both rows are changed to the acceleration that the boundary condition imposes. This is shown in Figure 7 for a case with four elements of order three where the movements of the line are both of its ends are imposed.

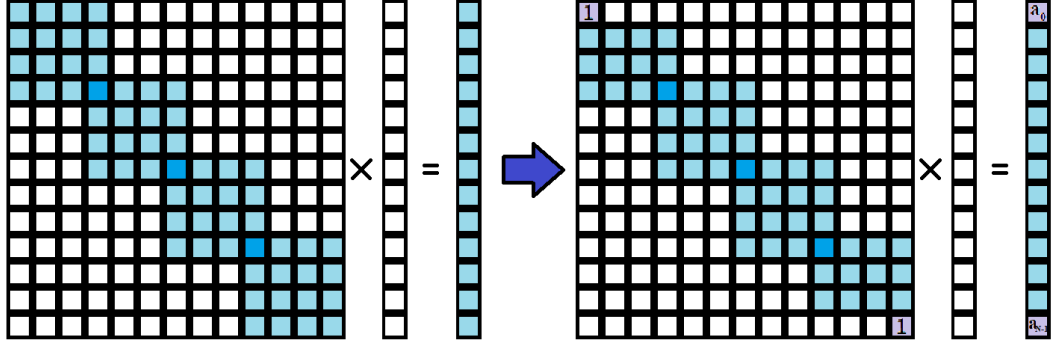


Figure 7: Boundary conditions modifications on the system.

Formally, this changes are expressed with the following pseudo-code:

```

A = M
A[0, 0] = 1
A[0, 1 : N - 1] = 0
A[N - 1, N - 1] = 1
A[N - 1, 0 : N - 2] = 0

B = -K* · F + M · f̂ + Γ
B[0, :] = â0
B[N - 1, :] = âN-1

```

where $\hat{\mathbf{a}}_0$ and $\hat{\mathbf{a}}_{N-1}$ are the accelerations imposed at nodes 0 and $N - 1$ respectively.

4.3.2 Multi-line joint conditions

An important feature of a line simulation software is to be able study the case where more than one line are connected to one point. An example of this situation is the crowfoot or delta mooring system, where there are two fairleads, one anchor, and three lines that join these three points to a common one, as shown in Figure 8.

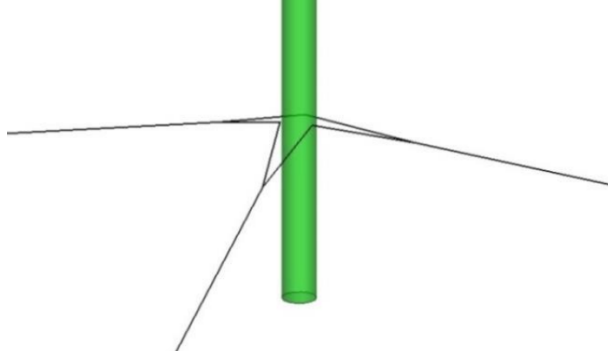


Figure 8: Crowfoot or delta mooring system.

Let there be a set of n lines converging to the same point, where $\{(\rho_0)_i\}$ are the linear weights of the different lines. In order to apply the joint boundary condition, first, the averages of the positions and the velocities of all the nodes that converge to the joint are imposed to those nodes, before computing the internal and external forces vectors. After that, the lines systems are solved without making any change to the equations of the nodes that are connected to the joint. Then, once the accelerations of all the nodes connected to the joint, $\{\hat{\mathbf{a}}_i\}$, are known, the acceleration of the joint, $\mathbf{a}_J$ is computed as:

$$\mathbf{a}_J = \frac{\sum_{i=1}^n [(\rho_0)_i \cdot \hat{\mathbf{a}}_i]}{\sum_{i=1}^n (\rho_0)_i}$$

Finally, this acceleration is imposed to all the nodes that converge to the joint.

4.3.3 Winch condition

Other important feature is to be able to simulate lines connected to winches, that may vary their length depending on the momentum applied with the winches engines, M , the properties of the winches (radius R and mass m) and the tension T of the lines at the winches. Figure 9 shows the scheme of a winch.

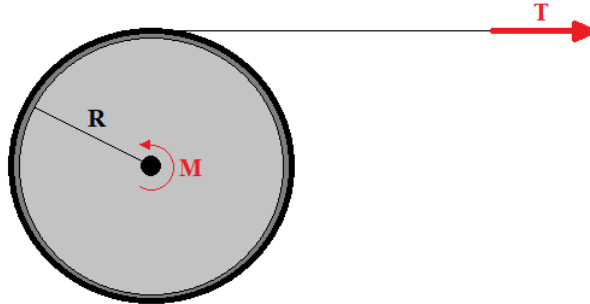


Figure 9: Winch scheme.

There are two aspects that we have to consider when there is a winch at the end of a line. The first one is how to compute how the winch rotates, and the second one is to compute how the winch rotation modifies the line length, and how the length variation affects the equations of the line.

In order to simulate the rotation of the winch, the Newton equation is applied in Equation 11.

$$I \cdot \alpha = T \cdot r - M \quad (11)$$

The left side of Equation 11 is the product of moment of inertia of the winch with the angular acceleration of the winch for rotations around the winch's axis. If the winch is considered to be a cylinder, the moment of inertia of the winch is $I = \frac{1}{2} \cdot m \cdot R^2$. The right side of the equation is the sum of the momentum that is applied to the winch, first the momentum that the line applies, considering that the radius of the winch and the tension of the line are perpendicular, and finally the momentum that the winch's engine applies. The signs are chosen some how that if the angle of rotation of the winch is positive, the line length increases, and if the winch's engine applies a positive momentum strong enough, the line length decreases.

Winch's time evolution is then described by the following initial value problem:

$$\begin{cases} \frac{\partial \mathbf{u}}{\partial t} = f(t) \\ \mathbf{u}(0) = \mathbf{u}_0 \end{cases}$$

With f being the ODE function, $\mathbf{u}_0$ known initial condition, and $\mathbf{u}$ the unknown. In this case the unknown will be:

$$\mathbf{u} = \begin{pmatrix} \theta \\ \omega \end{pmatrix}$$

where θ and ω are, respectively, the rotation and the angular velocity of the winch. The initial condition will be considered as zero:

$$\mathbf{u}_0 = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$$

and the ODE function is:

$$\begin{pmatrix} \omega \\ \alpha \end{pmatrix} = \frac{\partial \mathbf{u}}{\partial t} = f(t) = \begin{pmatrix} \omega \\ I^{-1} \cdot (T \cdot r - M) \end{pmatrix} \quad (12)$$

This ODE is integrated together with the line ODE, and the coupling between the winch and the line consists in that the line tension is used in the winch ODE and the winch rotation is used in the line ODE. This is because the line length is dependent on the winch rotation as it is shown in Equation 13.

$$L = L_0 + \theta \cdot R \quad (13)$$

where L is the line length, L_0 is the initial line length, and $theta$ and R , are respectively and as mentioned before, the winch rotation and radius. When the line length changes, instead of using the global derivation matrices $\mathbf{D}_n$ and the mass matrix $\mathbf{M}$ described in the previous section and used in the line ODE, the following matrices must be used instead:

$$\mathbf{D}_n \Rightarrow \left(\frac{L_0}{L} \right)^n \cdot \mathbf{D}_n$$

$$\mathbf{M} \Rightarrow \left(\frac{L}{L_0} \right) \cdot \mathbf{M}$$

With this, all the studied boundary conditions are already described, and only the last step is needed: the computation of the initial condition for all the line nodes.

4.4 Initial condition

For the initial condition, in general, a quasi-static (QS) method is used to find the positions of the line nodes. The velocity and acceleration of these nodes are considered as zero for the initial conditions. The QS method used is based in (7), and in this section a description of the method is presented.

The QS method ignores the dynamic effects on the line as inertia, friction, damping, or added mass and considers the line to be contained in a plane. The variables used for the QS method are:

ρ_w	water density,
A	line section
ρ_0	line mass per unit of length,
$\mathbf{g}$	gravity acceleration vector,
x_F	horizontal span between line ends,
z_F	vertical span between line ends,
H_F	horizontal tension at line's top support (usually the fairlead),
V_F	vertical tension at line's top support,
H_A	horizontal tension at line's bottom support (usually the anchor),
V_A	vertical tension at line's bottom support,
$\omega = (\rho_0 - \rho_w) \cdot A \cdot g$	line's underwater weight,
L	line length,
L_B	line length lying over the sea floor,
E	line Young's modulus and
C_B	line-ground friction coefficient.

Usually, the horizontal and vertical spans x_F and z_F are known and the unknowns are the tension values at both ends of the line.

On literature, the following expressions are found for the horizontal and vertical spans in terms of the top support tension:

If the line does not lie on the sea floor ($L_B = L - \frac{V_F}{\omega} \leq 0$):

$$x(H_F, V_F) = \frac{H_F}{\omega} \left[\ln \left(\frac{V_F}{H_F} + \sqrt{1 + \left(\frac{V_F}{H_F} \right)^2} \right) - \ln \left(\frac{V_F - \omega L}{H_F} + \sqrt{1 + \left(\frac{V_F - \omega L}{H_F} \right)^2} \right) \right] + \frac{H_F L}{EA}$$

$$z(H_F, V_F) = \frac{H_F}{\omega} \left[\sqrt{1 + \left(\frac{V_F}{H_F} \right)^2} - \sqrt{1 + \left(\frac{V_F - \omega L}{H_F} \right)^2} \right] + \frac{1}{EA} \left(V_F L - \frac{\omega L^2}{2} \right)$$

If part of the line lies on the sea floor⁴ ($L_B = L - \frac{V_F}{\omega} > 0$):

$$\begin{aligned} x(H_F, V_F) = L - \frac{V_F}{\omega} + \frac{H_F}{\omega} \ln \left(\frac{V_F}{H_F} + \sqrt{1 + \left(\frac{V_F}{H_F} \right)^2} \right) + \frac{H_F L}{EA} + \\ + \frac{C_B \omega}{2EA} \left[- \left(L - \frac{V_F}{\omega} \right)^2 + \left(L - \frac{V_F}{\omega} - \frac{H_F}{C_B \omega} \right) \max \left(L - \frac{V_F}{\omega} - \frac{H_F}{C_B \omega}, 0 \right) \right] \end{aligned}$$

$$z(H_F, V_F) = \frac{H_F}{\omega} \left[\sqrt{1 + \left(\frac{V_F}{H_F} \right)^2} - 1 \right] + \frac{1}{EA} \left(V_F L - \frac{\omega L^2}{2} \right)$$

Finding the pair (H_F, V_F) that yields the desired spans implies solving a non-linear system of equations, and this is done using a Newton-Raphson method. To do so, the function $\mathbf{F}(H_F, V_F)$ is defined as:

$$\mathbf{F}(H_F, V_F) = \begin{pmatrix} x(H_F, V_F) - x_F \\ z(H_F, V_F) - z_F \end{pmatrix}$$

in a way that the solution of the equation $\mathbf{F}(H_F, V_F) = 0$ is the solution of the non-linear system. Let $\mathbf{J}$ be the Jacobean matrix of $\mathbf{F}$. Let the unknown be expressed as $\mathbf{x} = (H_F, V_F)^T$. Let the initial guess be:

$$\mathbf{x}^0 = \left(\left| \frac{\omega x_F}{2\lambda_0} \right|, \frac{\omega}{2} \left[\frac{z_F}{\tanh(\lambda_0)} + L \right] \right)^T$$

⁴In this case, there is a "max" term, which differentiates if there is horizontal tension in the anchor or not. This should be considered as two separate cases for the Newton-Raphson method in order to compute the Jacobean matrix derivatives.

where:

$$\lambda_0 = \begin{cases} 10^6 & \text{for } x_F = 0 \\ 0.2 & \text{for } \sqrt{x_F^2 + z_F^2} \geq L \\ \sqrt{3 \left(\frac{L^2 - z_F^2}{x_F^2} - 1 \right)} & \text{otherwise.} \end{cases}$$

Then the solution is obtained updating the estimation of the solution with the expression:

$$\mathbf{x}^{k+1} = \mathbf{x}^k - \mathbf{J}^{-1}(\mathbf{x}^k) \cdot \mathbf{F}(\mathbf{x}^k)$$

until the convergence criteria $\|\mathbf{x}^{k+1} - \mathbf{x}^k\| < \text{tol}$ is met, where "tol" is the tolerance, or until the maximum number of iterations is reached.

Once the tension at the top support is obtained, the tension at the bottom can be easily computed with:

If the line does not lie on the sea floor:

$$H_A = H_F \quad V_A = V_F - \omega L$$

If part of the line lies on the sea floor:

$$H_A = \max(H_F - C_B \omega L_B, 0) \quad V_A = 0$$

Finally, the position and tension of any point of the line can be computed with the following expressions:

If the line does not lie on the sea floor:

$$x(s) = \frac{H_F}{\omega} \left[\ln \left(\frac{V_A + \omega s}{H_F} + \sqrt{1 + \left(\frac{V_A + \omega s}{H_F} \right)^2} \right) - \ln \left(\frac{V_A - \omega L}{H_F} + \sqrt{1 + \left(\frac{V_A - \omega L}{H_F} \right)^2} \right) \right] + \frac{H_F s}{EA}$$

$$z(s) = \frac{H_F}{\omega} \left[\sqrt{1 + \left(\frac{V_A + \omega s}{H_F} \right)^2} - \sqrt{1 + \left(\frac{V_A}{H_F} \right)^2} \right] + \frac{1}{EA} \left(V_A s - \frac{\omega s^2}{2} \right)$$

$$T_e(s) = \sqrt{H_F^2 + (V_A + \omega s)^2}$$

If part of the line lies on the sea floor:

$$x(s) = \begin{cases} s, & 0 \leq s \leq L_B - \frac{H_F}{C_B\omega} \\ s + \frac{C_B\omega}{2EA} \left[s^2 - 2 \left(L_B - \frac{H_F}{C_B\omega} \right) s + \left(L_B - \frac{H_F}{C_B\omega} \right) \max \left(L_B - \frac{H_F}{C_B\omega}, 0 \right) \right], & L_B - \frac{H_F}{C_B\omega} \leq s \leq L_B \\ L_B + \frac{H_F}{\omega} \ln \left[\frac{\omega(s-L_B)}{H_F} + \sqrt{1 + \left(\frac{\omega(s-L_B)}{H_F} \right)^2} \right] + \frac{H_F s}{EA} + \frac{C_B\omega}{2EA} \left[-L_B^2 + \left(L_B - \frac{H_F}{C_B\omega} \right) \max \left(L_B - \frac{H_F}{C_B\omega}, 0 \right) \right], & L_B \leq s \leq L \end{cases}$$

$$z(s) = \begin{cases} 0, & 0 \leq s \leq L_B \\ \frac{H_F}{\omega} \left[\sqrt{1 + \left(\frac{V_F}{H_F} \right)^2} - 1 \right] + \frac{1}{EA} \left(V_F L - \frac{\omega L^2}{2} \right), & L_B \leq s \leq L \end{cases}$$

$$T_e(s) = \begin{cases} \max(H_F + C_B\omega(s - L_B), 0), & 0 \leq s \leq L_B \\ \sqrt{H_F^2 + (\omega(s - L_B))^2}, & L_B \leq s \leq L \end{cases}$$

This is enough to compute the initial condition for most of the cases, but there are some exceptions in which the iterative method does not converge, or when the line system is not valid for the quasi-static approximation, for example, when there is a crowfoot joint or when the line is too tense. In this cases, an approximated initial condition will be used, considering the lines straight between their ends, and a previous simulation must be done to find the equilibrium state, which will be considered as the initial condition for other simulations.

Also if the case of study does not require a really precise mooring or towing lines simulation, or needs to be really fast, the QS method can be coupled with the floating bodies to run time evolution simulation.

5 Wind turbine

To be developed.

6 Oscillating water columns

To be developed.

7 Mechanical systems

(Alvaro) (6 DOF springs)

8 Time integration

(Alvaro) (BDF2 with variable step)

9 Coupling

(Alvaro) (Weak coupling)

References

- [1] A. C. Hindmarsh, ODEPACK, A Systematized Collection of ODE Solver, Editors: Stepleman, Carver, Peskin, Ames and Vichnevetsky: IMACS Transactions on Scientific Computation, 1983.
- [2] G. E. Karniadakis, Spectral/hp Element Methods for computational Fluid Dynamics. Second Edition. Oxford science Publications, 2004.
- [3] J. Azcona, Computational and Experimental Modelling of Mooring Lines Dynamics for Offshore Floating Wind Turbines. UPM. 2015
- [4] J. N. Newman, Marine Hydrodynamics, The Massachusetts Institute of Technology, 1977
- [5] J. Palm, C. Eskilsson and L. Bergdahl. An hp-adaptive discontinuous Galerkin method for modelling snap loads in mooring cables. Ocean Engineering, 144: 266-276, 2017.
- [6] O.M. Aamo and T.I. Fossen. Finite element modelling of mooring lines. Mathematics and Computers in Simulation, 53(4-6):415 – 422, 2000.
- [7] O.M. Faltinsen, Sea loads on ships and offshore structures. Cambridge ocean technology series. 1999
- [8] M. Hall and A. Goupee. Validation of a lumped-mass mooring line model with DeepCwind semisubmersible model test data. Ocean Engineering, 104:590 – 603, 2015.
- [9] M. R. Escalante, M.B. Rosales, R. Sampaio and T. Ritto, Reduced order model of a 3D cable using proper orthogonal decomposition. Mec. Comp.. 30. 1143-1158. 2011
- [10] P. Estévez. Matemáticas en ingeniería con MATLAB (Vol. 3). Universidad de Santiago de Compostela. 2001